

Oracle Object SQL

Complete Study Guide

Object Types • Inheritance • Methods • INSERT • Queries

Topics Covered

1. What is Object-Relational SQL?
2. Object Types — Theory & Syntax
3. Inheritance (UNDER keyword)
4. Member Functions & Method Bodies
5. Overriding & Overloading Methods
6. Object Tables & Nested Tables
7. INSERT into Object Tables
8. Querying with IS OF & VALUE()
9. Exam Questions — Fully Explained
10. Quick Reference Cheat Sheet

1. What is Oracle Object-Relational SQL?

Traditional SQL stores data in rows and columns (relational model). Oracle's Object-Relational model extends this by letting you define your own data types — called Object Types — that can have attributes (like columns) AND methods (like functions). This is similar to classes in Java or C++.

Key Idea

Think of an Object Type as a blueprint (like a class).
An Object Table stores rows that are instances of that blueprint.
Methods inside types let you add behaviour — not just data.

Relational SQL (Normal)	Object-Relational SQL (Oracle)
Tables with rows and columns	Object Types with attributes and methods

Relational SQL (Normal)	Object-Relational SQL (Oracle)
No inheritance between tables	Types can inherit using UNDER
Functions are standalone	Methods live inside the type
Only scalar values in columns	Columns can hold nested objects or tables

2. Object Types — Theory & Syntax

An Object Type is defined in two parts:

- Type Specification — defines attribute names and method signatures (like a header file).
- Type Body — contains the actual implementation of each method.

2.1 Creating an Object Type

The basic syntax is:

Syntax — Object Type

```
CREATE [OR REPLACE] TYPE type_name AS OBJECT (
    attribute1 datatype,
    attribute2 datatype,
    MEMBER FUNCTION method_name RETURN return_type
);
/

CREATE OR REPLACE TYPE BODY type_name AS
    MEMBER FUNCTION method_name RETURN return_type IS
    BEGIN
        -- implementation
    END;
END;
/
```

2.2 The Hospital Schema Explained

Your exam uses this schema. Read each type carefully:

The Hospital Schema

```
-- Step 1: Base type (abstract — cannot be instantiated directly)
CREATE TYPE Person_ty AS OBJECT (
    nic          CHAR(10),
    firstname    VARCHAR2(50),
    surname      VARCHAR2(50),
    MEMBER FUNCTION getID RETURN CHAR
) NOT FINAL NOT INSTANTIABLE;  -- key modifiers!
```

```
/

-- Step 2: Type body – default getID returns nic
CREATE OR REPLACE TYPE BODY Person_ty AS
  MEMBER FUNCTION getID RETURN CHAR IS
  BEGIN
    RETURN nic;
  END;
END;
/

-- Step 3: Doctor inherits Person_ty
CREATE TYPE Doctor_ty UNDER Person_ty (
  regNo      CHAR(10),
  specialty  VARCHAR2(25)
);
/

-- Step 4: Admission – a standalone object type
CREATE TYPE Admission_ty AS OBJECT (
  adDate      DATE,
  dischDate   DATE,
  illness     VARCHAR2(25),
  assgnDoctor REF Doctor_ty  -- a reference/pointer to a Doctor
);
/

-- Step 5: A nested table TYPE of admissions
CREATE TYPE Admission_ty_tlb AS TABLE OF Admission_ty;
/

-- Step 6: Patient inherits Person_ty
CREATE TYPE Patient_ty UNDER Person_ty (
  pid        CHAR(10),
  admissions Admission_ty_tlb -- nested table inside Patient
);
/

-- Step 7: Object Table – stores Person subtypes
CREATE TABLE People OF Person_ty;
```

2.3 Key Type Modifiers

Modifier	What it means
NOT FINAL	This type CAN be subtyped (extended). Without this, no UNDER is allowed.

Modifier	What it means
NOT INSTANTIABLE	You CANNOT directly create objects of this type. Must use a subtype.
UNDER parent_ty	This type inherits all attributes and methods of parent_ty.
REF Doctor_ty	Stores a reference (pointer) to a Doctor_ty object in the database.
TABLE OF type	Creates a nested table type — a collection of that type.

3. Inheritance — The UNDER Keyword

Inheritance allows a new type to automatically get all the attributes and methods of a parent type. The child type then adds its own extra attributes.

Rule to Remember

Child type fields come AFTER the parent's fields — always.

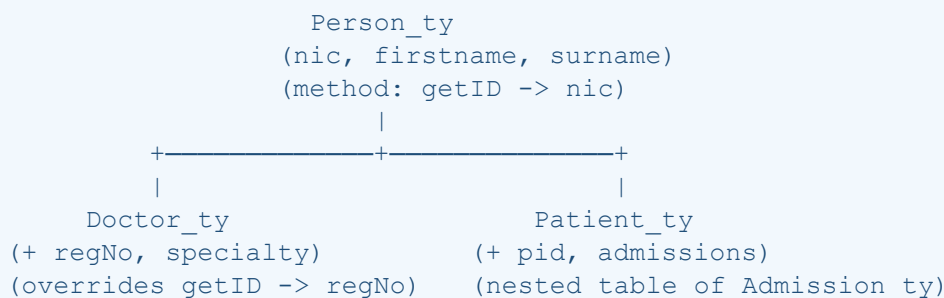
When constructing (inserting), always pass parent fields first, then child fields.

Doctor ty fields: nic, firstname, surname (from Person ty), then regNo, specialty (own).

Patient ty fields: nic, firstname, surname (from Person ty), then pid, admissions (own).

3.1 Inheritance Diagram

Inheritance Tree



4. Member Functions — Theory

A member function is a method that belongs to an object type. It can access the type's own attributes using SELF (implicit) — similar to 'this' in Java.

4.1 Defining a Method (Specification + Body)

Method Syntax

```
-- In the type specification:
MEMBER FUNCTION methodName(param datatype) RETURN return_datatype

-- In the type body:
MEMBER FUNCTION methodName(param datatype) RETURN return_datatype IS
BEGIN
    RETURN some_attribute; -- or any expression
END;
```

In your exam, Person_ty has getID which returns nic. The method body is:

Person_ty — getID

```
MEMBER FUNCTION getID RETURN CHAR IS
BEGIN
    RETURN nic; -- 'nic' refers to SELF.nic automatically
END;
```

5. Overriding vs Overloading Methods

Concept	What it means
Overriding	A child type redefines a method with the SAME name and SAME parameters. The child version runs instead of the parent's.
Overloading	A type has TWO methods with the SAME name but DIFFERENT parameters. Which one runs depends on what you pass.

5.1 Overriding getID in Doctor_ty

Problem: Doctor's getID should return regNo instead of nic.

Step 1 — Add the method declaration to the type using ALTER TYPE:

Step 1 — Declare Override

```
ALTER TYPE Doctor_ty
    ADD MEMBER FUNCTION getID RETURN CHAR CASCADE;
/
```

The CASCADE keyword means: automatically update any dependent types or tables.

Step 2 — Write the implementation in the type body:

Step 2 — Implement Override

```
CREATE OR REPLACE TYPE BODY Doctor_ty AS
  MEMBER FUNCTION getID RETURN CHAR IS
  BEGIN
    RETURN regNo;    -- returns regNo, not nic
  END;
END;
/
```

5.2 Overloading getID in Doctor_ty

Problem: Add a second getID(type VARCHAR2) method. If type='reg', return regNo. Otherwise return nic.

Both methods exist at the same time — Oracle picks which to call based on arguments passed.

Overloaded getID — Full Answer

```
-- Step 1: Add both overloaded method signatures
ALTER TYPE Doctor_ty
  ADD MEMBER FUNCTION getID(type VARCHAR2) RETURN CHAR CASCADE;
/

-- Step 2: Implement BOTH in the type body
CREATE OR REPLACE TYPE BODY Doctor_ty AS

  -- Original override (no params) -> returns regNo
  MEMBER FUNCTION getID RETURN CHAR IS
  BEGIN
    RETURN regNo;
  END;

  -- Overloaded version (1 param) -> conditional
  MEMBER FUNCTION getID(type VARCHAR2) RETURN CHAR IS
  BEGIN
    IF type = 'reg' THEN
      RETURN regNo;
    ELSE
      RETURN nic;
    END IF;
  END;

END;
/
```

6. Object Tables & Nested Tables

6.1 Object Tables

An Object Table stores objects of a given type (and its subtypes). Each row is a complete object.

Object Table

```
CREATE TABLE People OF Person_ty;
-- Each row in People is a Person_ty object (or a subtype like
Doctor/Patient)
```

6.2 Nested Tables

A Nested Table is a collection type — like an array stored inside an object. In Patient_ty, the admissions field holds multiple Admission_ty objects.

Nested Table Concept

```
-- The nested table TYPE is defined separately:
CREATE TYPE Admission_ty_tlb AS TABLE OF Admission_ty;
/

-- Then used as an attribute inside Patient_ty:
CREATE TYPE Patient_ty UNDER Person_ty (
  pid          CHAR(10),
  admissions Admission_ty_tlb  -- 0 or more Admission_ty objects
);
```

How to Use Nested Table in INSERT

When inserting, wrap nested table values in the nested table type constructor.

Example: Admission_ty_tlb(Admission_ty(...), Admission_ty(...))

If there are no admissions, use: Admission_ty_tlb()

Each inner object uses its own type constructor: Admission_ty(...)

7. INSERT into Object Tables — Full Theory

To insert into an Object Table, you use the type constructor — a function with the same name as the type. You pass all attributes in ORDER.

Golden Rules for INSERT

Rule 1: Use the subtype constructor (e.g. Doctor_ty(...), Patient_ty(...))

Rule 2: Pass parent attributes first, then child attributes.

Rule 3: Wrap nested tables using the nested table type constructor.

Rule 4: Use NULL for null references (REF fields with no value).

Rule 5: Dates are strings in format 'DD-MON-YY' (e.g. '2-MAR-03').

7.1 Inserting a Doctor

Doctor_ty field order: nic, firstname, surname (from Person_ty), regNo, specialty (own fields).

INSERT Doctor — Fully Annotated

```
INSERT INTO People VALUES (
  Doctor_ty(
    '569922008V',    -- nic      (from Person_ty)
    'Sampath',       -- firstname (from Person_ty)
    'Weerakoon',     -- surname  (from Person_ty)
    '1235567890',    -- regNo    (Doctor's own field)
    'pediatrician'   -- specialty (Doctor's own field)
  )
);
```

7.2 Inserting a Patient (with nested Admission)

Patient_ty field order: nic, firstname, surname (Person_ty), pid, admissions (own fields).

The admissions field uses the nested table constructor Admission_ty_tlb(...) containing Admission_ty(...) objects.

INSERT Patient — Fully Annotated

```
INSERT INTO People VALUES (
  Patient_ty(
    '55462008V',    -- nic      (from Person_ty)
    'Nimal',        -- firstname (from Person_ty)
    'Perera',       -- surname  (from Person_ty)
    '5678912367',   -- pid      (Patient's own field)
    Admission_ty_tlb( -- nested table constructor
      Admission_ty(  -- one admission record
        '2-MAR-03',  -- adDate
        '5-MAR-03',  -- dischDate
        'Diabetes',   -- illness
        null          -- assgnDoctor (REF, null means no doctor yet)
      )
    )
  )
);
```

8. Querying Object Tables

8.1 The VALUE() Function

When querying an object table, VALUE(alias) returns the entire object stored in a row. This is needed to filter by subtype.

VALUE() Function

```
-- Get all objects from People table
```



```
SELECT VALUE(p) FROM People p;

-- VALUE(p) returns the full object (Doctor_ty or Patient_ty or Person_ty)
```

8.2 IS OF — Filtering by Subtype

IS OF lets you select only rows that are of a specific subtype:

IS OF Filter

```
-- Get only doctors
SELECT VALUE(d) FROM People d
WHERE VALUE(d) IS OF (Doctor_ty);

-- Get only patients
SELECT VALUE(p) FROM People p
WHERE VALUE(p) IS OF (Patient_ty);
```

8.3 Calling Methods in SELECT

You can call a member function using the alias dot method notation:

Calling Methods in SELECT

```
-- Print IDs of all doctors using getID()
SELECT d.getID()          -- calls overridden getID -> returns regNo
FROM   People d
WHERE  VALUE(d) IS OF (Doctor_ty);

-- Print IDs using overloaded getID('reg')
SELECT d.getID('reg')     -- calls getID(VARCHAR2) -> returns regNo
FROM   People d
WHERE  VALUE(d) IS OF (Doctor_ty);
```

How Oracle Polymorphism Works

Person_ty.getID() returns nic.

Doctor_ty.getID() overrides it — returns regNo instead.

Oracle automatically calls the correct version based on the actual object type.

This is called polymorphism — same method name, different behaviour per type.

9. Exam Questions — Step-by-Step Explanations

Question 1 — Insert a Patient

Task: Insert Nimal Perera as a Patient into the People table.

How to approach it:

- Look at Patient_ty fields: inherited (nic, firstname, surname) + own (pid, admissions).
- The admissions field is a nested table — needs Admission_ty_tlb(Admission_ty(...)).
- assgnDoctor is null — just pass null for the REF field.

Answer — Question 1

```
INSERT INTO People VALUES (
  Patient_ty(
    '55462008V', 'Nimal', 'Perera', -- Person_ty fields
    '5678912367',                -- Patient pid
    Admission_ty_tlb(
      Admission_ty('2-MAR-03', '5-MAR-03', 'Diabetes', null)
    )
  )
);
```

Question 2 — Insert a Doctor

Task: Insert Dr. Sampath Weerakoon as a Doctor into the People table.

How to approach it:

- Look at Doctor_ty fields: inherited (nic, firstname, surname) + own (regNo, specialty).
- No nested tables — simple flat constructor call.

Answer — Question 2

```
INSERT INTO People VALUES (
  Doctor_ty(
    '569922008V', 'Sampath', 'Weerakoon', -- Person_ty fields
    '1235567890', 'pediatrician'          -- Doctor fields
  )
);
```

Question 3 — Override getID for Doctor_ty

Task: Write an overriding getID method for Doctor_ty that returns regNo.

How to approach it:

- Use ALTER TYPE to add the method declaration (same signature as parent's).
- Use CREATE OR REPLACE TYPE BODY to implement it, returning regNo instead of nic.

Answer — Question 3

```
-- Step 1: Add method signature to the type
ALTER TYPE Doctor_ty
```

```

    ADD MEMBER FUNCTION getID RETURN CHAR CASCADE;
/

-- Step 2: Implement the body
CREATE OR REPLACE TYPE BODY Doctor_ty AS
    MEMBER FUNCTION getID RETURN CHAR IS
    BEGIN
        RETURN regNo;
    END;
END;
/

```

Question 4 — Print IDs of All Doctors

Task: Use getID() and print the IDs of all doctors.

How to approach it:

- Query the People table with alias d.
- Use d.getID() in SELECT — calls overridden version for doctors.
- Filter with WHERE VALUE(d) IS OF (Doctor_ty).

Answer — Question 4

```

SELECT d.getID()
FROM   People d
WHERE  VALUE(d) IS OF (Doctor_ty);

```

Question 5 — Overloaded getID (with VARCHAR2 parameter)

Task: Add an overloaded getID(type VARCHAR2) to Doctor_ty. If type='reg', return regNo, otherwise return nic.

How to approach it:

- ALTER TYPE to add the new overloaded signature (different params = overload).
- In the TYPE BODY, include BOTH methods — the override (no params) AND the overload (1 param).
- Use IF/ELSE logic inside the overloaded method.

Answer — Question 5 — Full Overloaded Body

```

-- Step 1: Declare the overloaded method
ALTER TYPE Doctor_ty
    ADD MEMBER FUNCTION getID(type VARCHAR2) RETURN CHAR CASCADE;
/

-- Step 2: Implement both versions in the body

```

```

CREATE OR REPLACE TYPE BODY Doctor_ty AS

    MEMBER FUNCTION getID RETURN CHAR IS
    BEGIN
        RETURN regNo;
    END;

    MEMBER FUNCTION getID(type VARCHAR2) RETURN CHAR IS
    BEGIN
        IF type = 'reg' THEN
            RETURN regNo;
        ELSE
            RETURN nic;
        END IF;
    END;

END;
/

```

Using the Overloaded Method in a Query

Query with Overloaded getID

```

-- Using getID('reg') - returns regNo for doctors
SELECT d.getID('reg') AS Doctor_ID
FROM   People d
WHERE  VALUE(d) IS OF (Doctor_ty);

```

10. Quick Reference Cheat Sheet

What you need to do	Syntax Pattern
Create a base object type	CREATE TYPE name AS OBJECT (fields, MEMBER FUNCTION ...) NOT FINAL NOT INSTANTIABLE;
Create a child type	CREATE TYPE child_ty UNDER parent_ty (extra_fields);
Implement methods	CREATE OR REPLACE TYPE BODY type_name AS ... END;
Override a method	ALTER TYPE child_ty ADD MEMBER FUNCTION same_name RETURN type CASCADE;
Overload a method	ALTER TYPE type_name ADD MEMBER FUNCTION same_name(param type) RETURN type CASCADE;

What you need to do	Syntax Pattern
Create object table	<code>CREATE TABLE tbl_name OF object_type;</code>
Insert subtype object	<code>INSERT INTO tbl VALUES (SubType_ty(field1, field2, ...));</code>
Insert with nested table	<code>INSERT INTO tbl VALUES (Type_ty(..., NestedTbl_ty(Inner_ty(...))));</code>
Filter by subtype	<code>WHERE VALUE(alias) IS OF (SubType_ty)</code>
Call a method in SELECT	<code>SELECT alias.methodName() FROM table alias</code>
Return attribute in method	<code>BEGIN RETURN attribute_name; END;</code>
Conditional in method	<code>IF condition THEN RETURN x; ELSE RETURN y; END IF;</code>

Field Order Reference

Type	Field Order (in constructor)
Person_ty	nic, firstname, surname
Doctor_ty	nic, firstname, surname, regNo, specialty
Patient_ty	nic, firstname, surname, pid, admissions
Admission_ty	adDate, dischDate, illness, assgnDoctor

Most Common Mistakes to Avoid

Wrong field order in constructor — always parent fields first.
 Forgetting the nested table constructor — Admission_ty_tlb(...) is required.
 Missing CASCADE in ALTER TYPE — always add it.
 Writing getID without also keeping the override in the body when adding overload.
 Using IS OF without VALUE() — both are needed together in the WHERE clause.
 Forgetting the slash '/' after each DDL block in Oracle.